

UNIVERSIDADE TUIUTI DO PARANÁ

**BEATRIZ PIMENTEL DE MELLO
CAIO FEDERICO ESQUIVEL LOVERA ARZE
DENYSE PANZA CLEMENTE FERREIRA
DOUGLAS CLAYTON DA SILVA
FERNANDO BACH
LARISSA QUIRINO DOS SANTOS
LUCAS TOTEROL RODRIGUES**

**SISTEMA INTEGRADO DE MONITORAMENTO EM SERVIDOR LINUX:
MONITORAMENTO DE INFRAESTRUTURA E EVENTOS DE
SEGURANÇA**

**CURITIBA
2026**

**BEATRIZ PIMENTEL DE MELLO
CAIO FEDERICO ESQUIVEL LOVERA ARZE
DENYSE PANZA CLEMENTE FERREIRA
DOUGLAS CLAYTON DA SILVA
FERNANDO BACH
LARISSA QUIRINO DOS SANTOS
LUCAS TOTEROL RODRIGUES**

**SISTEMA INTEGRADO DE MONITORAMENTO EM SERVIDOR LINUX:
MONITORAMENTO DE INFRAESTRUTURA E EVENTOS DE
SEGURANÇA**

Projeto apresentado ao curso de Análise e Desenvolvimento de Sistemas, da Universidade Tuiuti do Paraná, como requisito avaliativo da disciplina de Projeto Interdisciplinar: Análise de Sistemas

Orientador: Prof. Luiz Altamir Corrêa Junior

**CURITIBA
2026**

RESUMO

A crescente dependência de ambientes computacionais torna a indisponibilidade e os incidentes de segurança fatores críticos para a continuidade dos serviços. Este trabalho apresenta a fundamentação teórica e o planejamento arquitetural de um sistema de monitoramento para servidores Linux, com foco na coleta de métricas de infraestrutura e na detecção de eventos básicos de segurança por meio da análise e logs. Com base em conceitos de monitoramento, observabilidade e segurança da informação, é proposta uma arquitetura cliente-servidor na qual agentes coletam dados de CPU, memória, disco e rede e os enviam a um servidor central para processamento e armazenamento. A solução inclui mecanismos de ingestão via API, persistência em banco de dados e visualização de informações, além de regras simples para detecção de padrões como tentativas de autenticação malsucedidas. Por se tratar de uma proposta em nível de especificação, não há implementação completa nem validação em ambiente real, o que implica limitações de escalabilidade e resiliência. Ainda assim, o trabalho demonstra a viabilidade conceitual da integração entre monitoramento de infraestrutura e análise de eventos de segurança.

Palavras-chave: Monitoramento de Sistemas; Servidores Linux; Análise de Logs; Observabilidade; Arquitetura de Software.

LISTA DE FIGURAS

FIGURA 1 – Topologia lógica do sistema de monitoramento.....	30
--	----

LISTA DE CÓDIGOS

CÓDIGO 1 – Exemplo de um log gerado por um Firewall.....	23
--	----

LISTA DE SIGLAS E ACRÔNIMOS

CPU	<i>Central Processing Unit</i> (Unidade Central de Processamento)
SRE	<i>Site Reliability Engineering</i> (Engenharia de Confiabilidade de Sites)
RAM	<i>Random Access Memory</i> (Memória de Acesso Aleatório)
I/O	<i>Input/Output</i> (Entrada/Saída)
R/W	<i>Read/Write</i> (Leitura/Escrita)
IOPS	<i>Input/Output Operations Per Second</i> (Operações de Entrada/Saída por Segundo)
JSON	<i>JavaScript Object Notation</i> (Notação de Objetos JavaScript)
JSONB	<i>JSON Binary</i> (Formato de armazenamento binário para JSON no PostgreSQL)
SNMP	<i>Simple Network Management Protocol</i> (Protocolo Simples de Gerenciamento de Redes)
MIB	<i>Management Information Base</i> (Base de Informações de Gerenciamento)
HDD	<i>Hard Disk Drive</i> (Unidade de Disco Rígido)
SSD	<i>Solid State Drive</i> (Unidade de Estado Sólido)
IDS	<i>Intrusion Detection System</i> (Sistema de Detecção de Intrusão)
SSH	<i>Secure Shell</i> (Terminal Seguro)
SYN	<i>Synchronize</i> (Pacote TCP de sincronização)
FIN	<i>Finish</i> (Pacote TCP de finalização)
LGPD	Lei Geral de Proteção de Dados
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
XML	<i>eXtensible Markup Language</i> (Linguagem de Marcação Extensível)
HTTP	<i>Hypertext Transfer Protocol</i> (Protocolo de Transferência de Hiper-texto)
TCP	<i>Transmission Control Protocol</i> (Protocolo de Controle de Transmissão)

SUMÁRIO

1	INTRODUÇÃO	8
2	MONITORAMENTO DE SISTEMAS	10
2.1	DEFINIÇÃO	10
2.2	IMPORTÂNCIA	10
2.3	MÉTRICAS DE MONITORAMENTO	11
2.3.1	CPU (Processador)	11
2.3.2	Memória RAM	11
2.3.3	Disco (Armazenamento)	11
2.3.4	Abordagens de Análise de Métricas	12
2.4	ALERTAS NO MONITORAMENTO	12
3	OBSERVABILIDADE	13
3.1	DIFERENÇA ENTRE MONITORAMENTO E OBSERVABILIDADE	13
3.2	OS TRÊS PILARES DA OBSERVABILIDADE	13
3.2.1	Métricas	13
3.2.2	Registros de Eventos (Logs)	14
3.2.3	Rastreamento Distribuído (Distributed Tracing)	14
4	COLETA DE DADOS DE UM ENDPOINT	15
4.1	COLETA DE DADOS EM SISTEMAS DE MONITORAMENTO	15
4.2	COLETA PERIÓDICA DE DADOS E SUAS UTILIDADES	16
4.3	ARMAZENAMENTO DE DADOS E SUAS ESTRUTURAS	16
4.4	TIPO DE REQUISIÇÃO DE COLETA DE DADOS	17
4.4.1	Exemplo didático: diferença entre polling e event-driven	17
5	VISUALIZAÇÃO E DASHBOARDS	19
6	AGENTES DE MONITORAMENTO	20
6.1	MODELO AGENTE - SERVIDOR	20
6.2	FERRAMENTAS DE MONITORAMENTO: ZABBIX E NAGIOS	21
7	LOGS E ANÁLISE DE EVENTOS	22
8	DETECÇÃO DE EVENTOS DE SEGURANÇA	24
8.1	DETECÇÃO DE EVENTOS DE SEGURANÇA E COMO SÃO DETECTADOS?	24
8.2	ATAQUES DE FORÇA BRUTA	25
8.3	ATAQUES DE FOOTPRINTING NA REDE (PORT SCAN)	25
8.4	A IMPORTÂNCIA DE SABER DETECTAR ESSES ATAQUES	26
9	COMUNICAÇÃO E ARQUITETURA	27
10	APLICAÇÃO DOS CONCEITOS NO PLANEJAMENTO DO SISTEMA DE MONITORAMENTO	29
10.1	VISÃO GERAL DA SOLUÇÃO PROPOSTA	29

10.2	ARQUITETURA FÍSICA E LÓGICA.....	30
10.3	COLETA DE DADOS E AGENTES DE MONITORAMENTO.....	31
10.4	ARMAZENAMENTO E ESTRUTURAÇÃO DOS DADOS.....	31
10.5	PROCESSAMENTO DE EVENTOS E ALERTAS	32
10.6	VISUALIZAÇÃO E DASHBOARD	32
10.7	LIMITAÇÕES DA SOLUÇÃO.....	33
11	CONSIDERAÇÕES FINAIS.....	34
	REFERÊNCIAS	35

1 INTRODUÇÃO

Atualmente, em ambientes corporativos, servidores desempenham papel essencial na sustentação de aplicações, serviços internos e comunicação em rede. A indisponibilidade de recursos, falhas operacionais ou incidentes de segurança podem impactar diretamente a continuidade dos serviços, tornando necessária a adoção de mecanismos que permitam acompanhar o estado dos sistemas de forma contínua. Nesse contexto, o monitoramento de sistemas possibilita a identificação de falhas, degradações de desempenho e indisponibilidades a partir da coleta e análise de dados operacionais. De acordo com a Fortinet (2026a), o monitoramento consiste na observação contínua da infraestrutura com o objetivo de detectar problemas e garantir a disponibilidade dos serviços.

Além disso, com o aumento da complexidade dos ambientes computacionais, observa-se a necessidade de ampliar a capacidade de análise desses sistemas, incorporando conceitos como observabilidade. Segundo China (2024), a observabilidade permite compreender o comportamento dos sistemas a partir de dados como métricas e registros de eventos. Nesse cenário, os logs assumem papel relevante na identificação de atividades suspeitas e na análise de incidentes de segurança, sendo considerados componentes fundamentais em processos de monitoramento, conforme destacado pelo Kent (2006).

Apesar da existência de diversas ferramentas no mercado, muitas soluções apresentam elevado nível de complexidade ou exigem infraestrutura robusta, o que dificulta sua adoção em ambientes de pequeno porte ou em contextos acadêmicos. Dessa forma, torna-se relevante investigar como os conceitos de monitoramento e análise de eventos podem ser aplicados na construção de soluções simplificadas e funcionais.

Diante desse contexto, este trabalho tem como problema de pesquisa a seguinte questão: como implementar um sistema capaz de monitorar métricas de infraestrutura e identificar eventos básicos de segurança em um ambiente de rede local?

O objetivo geral deste projeto é desenvolver um sistema de monitoramento para ambientes Linux, capaz de coletar métricas operacionais e identificar eventos de segurança a partir da análise de logs. Como objetivos específicos, destacam-se: (i) implementar mecanismos de coleta de dados nos dispositivos monitorados; (ii) desenvolver um servidor central para processamento e armazenamento das informações; (iii) identificar eventos de segurança com base em padrões conhecidos, como tentativas de autenticação malsucedidas e varreduras de rede; e (iv) disponibilizar uma interface para visualização dos dados e geração de alertas.

Para atingir esses objetivos, será adotada uma abordagem baseada em arquitetura cliente-servidor, com coleta de dados realizada por agentes e envio das

informações a um servidor central para processamento e armazenamento.

Este trabalho está organizado da seguinte forma: inicialmente, apresenta-se a fundamentação teórica relacionada ao tema. Em seguida, são descritos os aspectos de desenvolvimento do sistema proposto. Por fim, são apresentados os resultados obtidos e as considerações finais.

2 MONITORAMENTO DE SISTEMAS

Este capítulo dedica-se a explorar os fundamentos do monitoramento, iniciando pela sua definição formal e discutindo sua importância estratégica nas organizações modernas. Em seguida, são detalhadas as principais métricas utilizadas para aferir o desempenho da infraestrutura, com ênfase na abordagem metodológica USE (Utilização, Saturação e Erros), amplamente adotada para diagnóstico de gargalos. Por fim, são abordados os mecanismos de alerta, essenciais para a transição de uma postura reativa para uma atuação proativa na gestão de incidentes.

2.1 DEFINIÇÃO

O monitoramento de sistemas consiste no processo contínuo de coleta, exame e compreensão de dados sobre o funcionamento e o estado dos sistemas, assegurando que operem corretamente. Esse processo envolve a observação de métricas e eventos em tempo real, o que ajuda a identificar falhas, quedas de desempenho e comportamentos anormais.

De acordo com Tanenbaum e Bos (2015), vigiar constantemente os componentes do sistema é crucial para administrar sistemas operacionais de maneira eficaz, especialmente no que se refere ao uso de CPU, memória e dispositivos de entrada e saída. Adicionalmente, pesquisas como a de Aceto *et al.* (2013) ressaltam que o monitoramento desempenha um papel fundamental em ambientes distribuídos, ao permitir a constante coleta de dados sobre o estado do sistema e auxiliar métodos de gestão e adequação flexível.

2.2 IMPORTÂNCIA

A importância do monitoramento de sistemas está diretamente relacionada à necessidade de garantir disponibilidade, desempenho e confiabilidade dos serviços de TI. Nos cenários atuais, nos quais as aplicações dependem fortemente da infraestrutura computacional, a ausência de monitoramento prejudica significativamente a agilidade para solucionar erros.

De acordo com a IBM (2022a), o monitoramento ajuda a identificar problemas antes que eles afetem os usuários finais, diminuindo o tempo de inatividade do sistema e aumentando a eficácia das operações. Conforme discutido por Beyer *et al.* (2016) no modelo de Engenharia de Confiabilidade do Google (SRE), o monitoramento contínuo possibilita acompanhar a condição do sistema em tempo real, oferecendo dados cruciais para o planejamento de capacidade e a melhoria do desempenho por meio do estudo de padrões ao longo do tempo.

Outro ponto relevante é o auxílio na tomada de decisão. A partir dos dados coletados, gestores podem identificar padrões de uso, prever demandas futuras e ajustar a infraestrutura de forma mais eficiente. Desse modo, o monitoramento torna-se mais do que uma ferramenta técnica, configurando-se como um recurso estratégico nas empresas, uma vez que o uso de dados analíticos contribui para decisões mais assertivas e competitivas (DAVENPORT, 2006).

2.3 MÉTRICAS DE MONITORAMENTO

As métricas são dados numéricos que servem para medir o desempenho dos recursos computacionais. Segundo Gregg (2020), a análise de métricas como uso de CPU, memória e armazenamento é fundamental para identificar gargalos e compreender o comportamento de sistemas computacionais, o que ajuda a fazer diagnósticos melhores e otimizar de forma eficaz.

2.3.1 CPU (Processador)

O acompanhamento do uso do processador é crucial, pois está diretamente relacionado à capacidade de processamento do sistema. Essa métrica geralmente é expressa em percentual, indicando o quanto do recurso está sendo utilizado em determinado momento. Segundo a Amazon Web Services (2023), altos níveis de utilização da CPU podem indicar sobrecarga, enquanto valores constantemente baixos podem apontar subutilização de recursos. Além disso, a análise dessa métrica ao longo do tempo permite identificar gargalos e necessidades de escalabilidade.

2.3.2 Memória RAM

A memória RAM atua como um espaço de trabalho temporário, guardando as informações que os programas em uso necessitam, sendo crucial para que os computadores operem sem engasgos. O monitoramento da memória é importante para prevenir situações como lentidão e travamentos. De acordo com Tanenbaum e Bos (2015), a insuficiência de memória pode levar ao uso excessivo de memória virtual (*swap*), o que impacta negativamente o desempenho do sistema. Métricas como memória utilizada, memória livre e uso de *swap* são amplamente utilizadas para avaliar a eficiência do uso desse recurso.

2.3.3 Disco (Armazenamento)

O monitoramento de disco envolve a análise das operações de entrada e saída (E/S), englobando a gravação e o acesso a dados. Esse tipo de métrica é essencial para identificar gargalos relacionados ao armazenamento. Segundo Barrett *et al.*

(2016), métricas como IOPS (operações de entrada/saída por segundo), latência e taxa de transferência são fundamentais para avaliar o desempenho do subsistema de armazenamento. Problemas de disco podem impactar diretamente o tempo de resposta das aplicações e a experiência do usuário.

2.3.4 Abordagens de Análise de Métricas

Uma abordagem amplamente utilizada na análise de métricas é o método USE (*Utilization, Saturation, Errors*), proposto por Brendan Gregg. Esse método consiste na análise de três aspectos principais dos recursos computacionais:

- **Utilização:** nível de uso do recurso ao longo do tempo;
- **Saturação:** presença de filas ou sinais de sobrecarga, indicando que o recurso está no limite de sua capacidade;
- **Erros:** falhas ocorridas durante a operação do sistema.

De acordo com Gregg (2013), essa abordagem é eficaz para a identificação sistemática de problemas de desempenho, pois permite uma análise estruturada e abrangente dos principais recursos do sistema.

2.4 ALERTAS NO MONITORAMENTO

Os alertas são mecanismos essenciais dentro do monitoramento de sistemas, responsáveis por notificar automaticamente administradores ou outros sistemas quando determinadas condições são atendidas, como a ultrapassagem de limites previamente definidos. A configuração de alertas possibilita uma resposta rápida a incidentes, reduzindo o impacto de falhas e melhorando a disponibilidade dos serviços (GOOGLE CLOUD, 2023). Esses alertas podem ser configurados com base em limites fixos, padrões históricos ou técnicas de detecção de anomalias.

Sistemas modernos de monitoramento utilizam diferentes tipos de alertas, incluindo:

- Alertas baseados em limiar (*threshold*);
- Alertas baseados em comportamento;
- Alertas baseados em anomalias.

A correta configuração de alertas é fundamental para evitar problemas como o excesso de notificações, conhecido como *alert fatigue*, que pode reduzir a efetividade do monitoramento e comprometer a resposta a incidentes (IBM, 2022b).

3 OBSERVABILIDADE

Diferente do monitoramento, que foca no estado do sistema, a observabilidade é uma característica essencial do mesmo. Conforme Majors *et al.* (2022), isso possibilita que o desenvolvedor de software rastreie um problema (o sintoma) até a sua origem, sem necessitar de hipóteses iniciais.

3.1 DIFERENÇA ENTRE MONITORAMENTO E OBSERVABILIDADE

Embora os termos sejam frequentemente utilizados como sinônimos, a literatura técnica moderna estabelece distinções fundamentais entre eles. Conforme as práticas de Engenharia de Confiabilidade de Beyer *et al.* (2016), o monitoramento é o processo de coletar, agregar e analisar métricas em tempo real para entender o estado de um sistema em relação a um conjunto de indicadores pré-definidos. Essa ação se concentra no que está acontecendo, sendo muito importante para o funcionamento básico.

Por outro lado, a observabilidade, definida por Schwartz (2017), é o quanto é possível saber sobre o funcionamento interno de um sistema apenas observando seus sinais externos. Enquanto o monitoramento age quando algo já deu errado e se baseia em falhas já conhecidas, a observabilidade é proativa e antecipa problemas. Conforme explicam Majors *et al.* (2022), a observabilidade ajuda os engenheiros a investigar problemas que nunca apareceram antes (os chamados *unknown unknowns*), permitindo que eles naveguem pelos dados de telemetria sem precisar criar alertas para cada possível erro.

3.2 OS TRÊS PILARES DA OBSERVABILIDADE

A base de um sistema observável é construída sobre três categorias diferentes de dados de telemetria: métricas, logs e *traces*. Como demonstrado por Sigelman *et al.* (2010), o segredo para entender tudo o que acontece em sistemas complexos está em juntar esses dados. As métricas mostram se algo está errado usando números gerais, enquanto os logs guardam um histórico completo e detalhado de tudo que aconteceu. E, para completar, os *traces* mostram o caminho que cada solicitação faz, desde o início até o fim, ajudando a encontrar exatamente onde está o problema ou a lentidão em um sistema complicado (MAJORS *et al.*, 2022).

3.2.1 Métricas

As métricas são representações numéricas de dados agregados ao longo de intervalos de tempo. Segundo a especificação do OpenTelemetry (2023), elas são otimi-

zadas para armazenamento e consulta rápida, sendo o pilar ideal para o monitoramento de saúde do sistema e acionamento de alertas.

3.2.2 Registros de Eventos (Logs)

Diferente das métricas, os logs consistem em anotações textuais distintas de ocorrências em um instante determinado. Para Majors *et al.* (2022), o log formatado (como no padrão JSON) apresenta maior utilidade, já que possibilita a busca por características particulares dentro da comunicação por meio de aplicativos de análise. Nos aprofundaremos mais nos logs no capítulo 7.

3.2.3 Rastreamento Distribuído (Distributed Tracing)

O rastreamento é o pilar que conecta os pontos em uma arquitetura de microsserviços, mantendo o registro do caminho de cada solicitação enquanto ela passa por diferentes partes da estrutura. A base técnica para este pilar é o artigo “Dapper” (SIGELMAN *et al.*, 2010), que introduziu os conceitos de *span* e *trace ID*. Além disso, Cederberg (2021) ressalta que, em ambientes com microsserviços, o rastreamento distribuído é essencial para enxergar como os serviços autônomos se relacionam no tempo, permitindo a identificação de problemas interligados que passariam despercebidos em registros isolados.

4 COLETA DE DADOS DE UM ENDPOINT

A coleta de dados de um *endpoint*¹ e o modelo de comunicação são etapas fundamentais na arquitetura de sistemas, pois permitem a obtenção contínua de informações e a transferência de dados entre cliente e servidor, que será mais explicado no capítulo 9. Esses dados representam atualizações e mudanças de estado, sendo essenciais para o gerenciamento em tempo real das aplicações.

4.1 COLETA DE DADOS EM SISTEMAS DE MONITORAMENTO

A coleta de dados é uma etapa fundamental em sistemas de monitoramento, especialmente, mas não se limitando a ambientes Linux, pois permite a obtenção contínua de informações sobre o desempenho e o estado dos recursos computacionais. Esses dados incluem métricas como uso de CPU, memória, disco e tráfego de rede, sendo essenciais para o gerenciamento da infraestrutura.

No contexto de monitoramento de redes, o uso de agentes é uma abordagem amplamente adotada. Conforme descrito por Santos (2019, p. 14), “o agente é responsável por coletar as informações do dispositivo monitorado e disponibilizá-las ao sistema gerenciador”. Esse modelo permite que os dados sejam capturados diretamente no sistema operacional e enviados para análise centralizada.

Um dos principais protocolos utilizados nesse processo é o SNMP (*Simple Network Management Protocol*), que possibilita a coleta estruturada de informações por meio de objetos organizados em uma base denominada MIB (*Management Information Base*). De acordo com Case *et al.* (2002), o SNMP foi desenvolvido com o objetivo de facilitar o gerenciamento de redes, permitindo a leitura e manipulação de informações de dispositivos de forma padronizada.

Além disso, a arquitetura de monitoramento envolve três componentes principais: o agente (capítulo 6), o dispositivo monitorado (*endpoint*) e o gerenciador(servidor). O agente coleta os dados localmente, enquanto o gerenciador é responsável por centralizar e analisar essas informações. Em sistemas práticos, como os que utilizam ferramentas de monitoramento, essa estrutura é essencial para garantir eficiência e escalabilidade.

A coleta de dados pode ocorrer por diferentes métodos, sendo os principais o *polling* e o orientado a eventos, que veremos mais adiante. Conforme apontado por Souza e Oliveira (2020, p. 6), “o monitoramento pode ser realizado por verificações periódicas (*polling*) ou por notificações automáticas enviadas pelos dispositivos”. Essa combinação permite maior confiabilidade na detecção de falhas.

¹ Endpoint refere-se a um ponto final de comunicação em uma rede. O termo pode designar tanto um dispositivo participante, como um servidor ou estação de trabalho, quanto um ponto específico de acesso a um serviço, como uma URL em uma API.

Comentado [LM1]: @Lucas Toterol Rodrigues muito bom essa definição. Isso é algo que galera demora para compreender

4.2 COLETA PERIÓDICA DE DADOS E SUAS UTILIDADES

A coleta periódica de dados consiste na obtenção de informações em intervalos regulares de tempo, sendo uma prática essencial em sistemas de monitoramento. Essa abordagem permite a criação de históricos que auxiliam na análise do comportamento dos sistemas ao longo do tempo.

De acordo com Ferreira (2020, p. 22), “a coleta contínua de dados possibilita o acompanhamento da evolução dos recursos computacionais e a identificação de padrões de uso”. Essa característica é fundamental para ambientes que exigem alta disponibilidade e desempenho.

Uma das principais vantagens da coleta periódica é a capacidade de detectar anomalias e prever falhas antes que causem impactos significativos. Nesse sentido, a análise de dados históricos desempenha papel essencial. Conforme destacado por Lima (2021, p. 35), “a análise de dados coletados ao longo do tempo permite identificar comportamentos fora do padrão e agir de forma preventiva”.

Além disso, a coleta periódica contribui para o planejamento de capacidade, pois fornece informações sobre o crescimento do uso de recursos. Segundo Ferreira (2020, p. 24), esses dados “auxiliam no dimensionamento adequado da infraestrutura e na tomada de decisões estratégicas”.

Outro aspecto importante refere-se à definição da frequência de coleta. Intervalos muito curtos podem gerar sobrecarga no sistema, enquanto intervalos muito longos podem comprometer a precisão das análises. Conforme observado por Santos (2019, p. 18), “a escolha da periodicidade de coleta deve equilibrar desempenho e qualidade das informações obtidas”.

Por fim, a coleta periódica permite a geração de relatórios, auditorias e análises históricas, sendo essencial para a gestão eficiente dos sistemas computacionais.

Comentado [LM2]: @Lucas Toterol Rodrigues bem o que questionou ontem sobre o intervalo rs

4.3 ARMAZENAMENTO DE DADOS E SUAS ESTRUTURAS

O armazenamento de dados consiste na retenção de informações digitais em meios físicos, sendo uma prática essencial em sistemas computacionais para as operações de software. Essa abordagem permite a criação de um ecossistema sólido que auxilia na consulta, análise e uso das informações pelas aplicações.

De acordo com Susnjara e Smalley (2025), “armazenamento de dados refere-se a mídias magnéticas, ópticas ou mecânicas que registram e preservam informações digitais para operações contínuas ou futuras”. Essa característica física, baseada em memórias RAM, HDDs e SSDs, é fundamental para ambientes que exigem alta disponibilidade e desempenho.

Uma das principais decisões no armazenamento é a escolha do modelo de banco de dados, que organiza os registros para que não causem impactos significativos

Comentado [LM3]: @Lucas Toterol Rodrigues aqui me parece que falta o modo "eletronico" de armazenamento que seria o do SSD por exemplo

Comentado [LM4R3]: Talvez por ser citacao nao tenha mesmo...sei la rs

na aplicação. Nesse sentido, os bancos relacionais desempenham papel essencial. Conforme destacado pela Amazon Web Services (2026), “os bancos de dados relacionais armazenam dados em formato de tabela, consistindo em linhas e colunas”.

Além disso, o formato físico de armazenamento contribui para a transferência e a confiabilidade da infraestrutura, como ocorre no armazenamento em blocos. Segundo Susnjara e Smalley (2025), nesse formato os dados “são armazenados como peças separadas, cada uma com um identificador único”.

Outro aspecto importante refere-se à escolha da manutenção da infraestrutura, dividida entre modelos autogerenciados e serviços na nuvem. Modelos autogerenciados exigem grande esforço técnico operacional, enquanto a nuvem traz conveniência. Conforme observado pela Amazon Web Services (2026), “os benefícios de usar um serviço gerenciado incluem a redução do tempo gasto em gerenciamento e manutenção da infraestrutura e maior segurança”.

Por fim, o correto armazenamento físico e lógico permite a geração de valor, consultas estruturadas e integração de serviços, sendo essencial para a gestão eficiente das plataformas e dos sistemas corporativos.

4.4 TIPO DE REQUISIÇÃO DE COLETA DE DADOS

No contexto de verificação de atualizações, o uso de *polling* é uma abordagem amplamente adotada. Conforme Rios (2024), o *polling* “refere-se a uma técnica de comunicação onde um cliente solicita regularmente – ou em intervalos regulares – informações de um servidor”. Esse modelo permite que os dados sejam verificados de maneira ativa pela aplicação cliente, por meio de requisições cíclicas.

Um dos principais modelos utilizados para otimizar esse processo é a arquitetura orientada a eventos (*event-driven*), que possibilita uma comunicação estruturada reagindo a mudanças de estado. De acordo com a Amazon Web Services (2025), a arquitetura baseada em eventos “usa eventos para acionar e comunicar entre serviços desacoplados”, permitindo a manipulação de informações de forma assíncrona e independente.

Além disso, a arquitetura orientada a eventos envolve três componentes principais: produtores, roteadores (*brokers*) e consumidores. O produtor emite os eventos; o roteador é responsável por centralizar, filtrar e distribuir essas mensagens aos consumidores. Em sistemas práticos modernos, essa estrutura é essencial para garantir eficiência, tolerância a falhas e escalabilidade.

4.4.1 Exemplo didático: diferença entre polling e event-driven

Para compreender melhor a diferença entre os dois modelos, imagine o controle de acesso a dois edifícios diferentes:

Comentado [LM5]: @Lucas Toterol Rodrigues sera que aqui nao seria em objetos ao inves de blocos?

Comentado [LT6R5]: De acordo com nossas fontes consultadas, é pra ser em bloco mesmo.

Comentado [LM7]: Os brokers que voce me perguntou certo dia, lembra? Ate se tiver como fazer uma nota de rodapé explicando, acho que seria legal @Lucas Toterol Rodrigues

Comentado [LM8R7]: Tambem nao sei se “roteador” é o melhor nome, pois confunde com o que vemos em redes

Comentado [LT9R7]: Boa ideia! Fazer parecido com o que fizemos em endpoint

Modelo polling – Prédio com detecção facial automática: O sistema de câmeras e sensores funciona continuamente, verificando o rosto de quem se aproxima a cada fração de segundo. Mesmo quando não há ninguém, o sistema segue “perguntando” ao servidor: “Alguém quer entrar?” – centenas de vezes por minuto. Isso gera muitas verificações desnecessárias (chamadas vazias), mas o acesso é liberado imediatamente quando uma pessoa autorizada é detectada.

Modelo event-driven – Prédio com porteiro eletrônico: O sistema fica ocioso até que alguém toque a campainha (o evento). Só então o porteiro (o roteador) é acionado, verifica a autorização e libera a entrada. Nenhuma verificação ocorre enquanto não há um evento. Isso elimina chamadas vazias, mas a resposta depende da ocorrência do evento.

Em resumo: no *polling* o cliente pergunta repetidamente (ativo); no *event-driven* o servidor avisa quando algo muda (reativo). A escolha depende do cenário: *polling* é simples e fácil de implementar, enquanto *event-driven* é mais eficiente e escalável para sistemas com muitos eventos esporádicos.

A comunicação de dados pode ocorrer por diferentes métodos, sendo os principais o *polling* e o modelo orientado a eventos. Conforme apontado pela Microsoft (2026), no modelo orientado a eventos, “os eventos são entregues quase em tempo real, para que os consumidores possam responder imediatamente aos eventos assim que ocorrerem”. Essa combinação elimina as chamadas vazias e garante maior agilidade no fluxo de dados da rede.

5 VISUALIZAÇÃO E DASHBOARDS

Os *dashboards*, também conhecidos como painéis de informação, são ferramentas fundamentais na gestão moderna, pois permitem organizar, analisar e visualizar dados de forma clara, interativa e consolidada em um único ambiente (CABALLERO, 2024). Sua principal função é transformar dados brutos em *insights* estratégicos, facilitando a compreensão de informações complexas e apoiando a tomada de decisão de maneira rápida, precisa e baseada em dados concretos (ROLIM, 2020).

Essas ferramentas oferecem uma visão abrangente e atualizada ao centralizar dados provenientes de diferentes fontes, possibilitando o acompanhamento do desempenho e dos resultados organizacionais em tempo real (CABALLERO, 2024). Por meio de elementos visuais como gráficos de barras, gráficos de setores, mapas interativos e cartões de métricas, os *dashboards* tornam a interpretação das informações mais intuitiva, permitindo identificar padrões, tendências e anomalias com maior facilidade (CAMPOS, 2023). Além disso, a interatividade, com filtros, menus e navegação por abas, possibilita que o usuário explore diferentes recortes dos dados, enquanto recursos como o *drill-down* permitem aprofundar a análise e obter informações mais detalhadas (CAMPOS, 2023).

Outro aspecto relevante é a capacidade de sintetizar grandes volumes de dados, reduzindo a complexidade e evitando sobrecarga de informações, o que é especialmente importante em contextos com dados heterogêneos (ROLIM, 2020). A integração de análises multidimensionais e geoespaciais também amplia a compreensão dos dados, permitindo visualizar relações espaciais e temporais de forma clara (CABALLERO, 2024).

Os *dashboards* ainda contribuem diretamente para a eficiência operacional, ao possibilitar o monitoramento contínuo e em tempo real de indicadores, facilitando a identificação imediata de problemas e a adoção de ações corretivas (CABALLERO, 2024). Além disso, promovem vantagem competitiva ao otimizar o tempo de análise, reduzir custos e melhorar a qualidade dos serviços, substituindo relatórios tradicionais por visualizações dinâmicas e interativas (ROLIM, 2020).

Por fim, destacam-se pela acessibilidade, permitindo que diferentes perfis de usuários, mesmo sem grande conhecimento técnico, consigam interpretar dados e extrair *insights* relevantes para embasar decisões estratégicas (CABALLERO, 2024). Dessa forma, os *dashboards* atuam como uma ponte entre a coleta de dados e a ação, simplificando a análise de informações complexas e tornando a gestão mais eficiente, ágil e orientada por dados (CAMPOS, 2023).

Comentado [LM10]: Excelente

6 AGENTES DE MONITORAMENTO

Conforme falado no capítulo 4 agentes de monitoramento são programas instalados nos dispositivos com a finalidade de coletar informações sobre o estado e o desempenho dos sistemas computacionais. Esses agentes atuam diretamente no sistema operacional, capturando métricas como uso de CPU, memória, disco e processos em execução, sendo essenciais para o acompanhamento da infraestrutura de TI.

De acordo com Santos (2019, p. 13), “os agentes de monitoramento são programas executados nos dispositivos que coletam informações e as disponibilizam para sistemas gerenciadores”. Dessa forma, eles funcionam como intermediários entre os dispositivos monitorados e os sistemas centrais, garantindo que os dados sejam coletados de forma organizada e confiável.

O funcionamento desses agentes baseia-se na coleta contínua ou sob demanda de dados, que são posteriormente enviados para um servidor central para análise. Conforme destacado por Ferreira (2020, p. 21), “os agentes realizam a coleta contínua de dados do sistema e os encaminham para análise”, permitindo o acompanhamento do comportamento dos recursos ao longo do tempo e a identificação de possíveis falhas.

Além disso, os agentes podem operar em diferentes modos, como ativo e passivo. No modo passivo, respondem a requisições do sistema gerenciador (*polling*), enquanto no modo ativo enviam informações automaticamente quando ocorrem eventos relevantes. Segundo Souza e Oliveira (2020, p. 5), “os sistemas de monitoramento podem operar com verificações periódicas ou envio automático de alertas”, o que aumenta a eficiência na detecção de problemas.

Por fim, destaca-se que os agentes frequentemente utilizam protocolos padronizados, como o **SNMP**, para comunicação com os sistemas de monitoramento. Conforme estabelecido por Case *et al.* (2002), esses protocolos permitem a padronização da coleta de dados, facilitando a interoperabilidade entre diferentes dispositivos e tornando o monitoramento mais escalável e eficiente.

6.1 MODELO AGENTE - SERVIDOR

Enquanto no modelo cliente-servidor o cliente não compartilha nenhum de seus recursos com o servidor, é ele quem inicia a comunicação solicitando alguma função ao servidor que fica aguardando as requisições. Com o agente a comunicação passa a ser orientada a eventos: o agente só contacta o servidor quando há algo relevante a reportar.

Wooldridge e Jennings (1995, p. 116) identificam seis propriedades do agente: a **autonomia**, em que os agentes operam sem a intervenção direta de humanos ou

Comentado [LM11]: @Lucas Toterol Rodrigues aqui acho que caberia uma definição de SNMP se já não tiver

Comentado [LM12]: @Lucas Toterol Rodrigues 2 vírgulas

outros e têm algum tipo de controle sobre suas ações e estado interno; a **habilidade social**, na qual os agentes interagem com outros agentes ou possivelmente humanos por meio de algum tipo de linguagem de comunicação; a **reatividade**, onde os agentes percebem seu ambiente, que pode ser o mundo físico, um usuário, uma coleção de outros agentes ou a Internet, e respondem em tempo hábil às mudanças que ocorrem nele; e a **proatividade**, em que os agentes não agem simplesmente em resposta ao seu ambiente, sendo capazes de exibir comportamento direcionado a objetivos e tomar a iniciativa.

Comentado [LM13]: @Lucas Toterol Rodrigues nao sabia que havia essa disticao do modelo de cliente servidor para agente servidor

6.2 FERRAMENTAS DE MONITORAMENTO: ZABBIX E NAGIOS

Diversas ferramentas utilizam agentes de monitoramento para coletar dados e acompanhar o funcionamento dos sistemas, sendo o Zabbix e o Nagios duas das soluções mais utilizadas tanto no meio acadêmico quanto no mercado. Essas ferramentas exemplificam, na prática, a aplicação dos conceitos de monitoramento em ambientes reais.

O Nagios é uma ferramenta consolidada que permite monitorar *hosts*, serviços e dispositivos de rede por meio de agentes e *plugins*. Conforme afirmam Souza e Oliveira (2020, p. 7), “o Nagios permite o monitoramento de *hosts* e serviços, utilizando *plugins* e agentes para coleta de dados em diferentes ambientes”, o que demonstra sua flexibilidade e ampla aplicabilidade.

Uma característica importante do Nagios é a possibilidade de operar com verificações ativas e passivas, permitindo adaptar o monitoramento conforme as necessidades do ambiente. Essa abordagem possibilita tanto a coleta periódica de dados quanto o envio de alertas automáticos, contribuindo para a rápida identificação de falhas.

O Zabbix, por sua vez, destaca-se pelo monitoramento em tempo real e pela capacidade de lidar com grandes volumes de dados. Ele utiliza agentes instalados nos dispositivos para coletar informações detalhadas sobre o desempenho do sistema. Segundo Ferreira (2020, p. 25), o Zabbix “permite monitoramento em tempo real de diversos parâmetros, com suporte a coleta distribuída de dados”, o que o torna adequado para ambientes mais complexos.

Dessa forma, tanto o Zabbix quanto o Nagios demonstram a importância dos agentes de monitoramento na coleta e análise de dados. Essas ferramentas permitem a geração de alertas, relatórios e análises de desempenho, contribuindo para a gestão eficiente dos sistemas e garantindo maior confiabilidade na infraestrutura de TI.

7 LOGS E ANÁLISE DE EVENTOS

Os logs são registros de eventos gerados por sistemas, aplicações e dispositivos ao longo de seu funcionamento, nos quais cada entrada representa uma ocorrência específica. Segundo Kent (2006), “um log é um registro dos eventos que ocorrem nos sistemas e redes de uma organização”. De acordo com a IBM (2025a), esses registros constituem dados essenciais para as operações de TI, pois fornecem informações relevantes sobre desempenho e segurança.

A importância dos logs está relacionada à visibilidade que proporcionam e à capacidade de reconstruir ações dentro do ambiente. Todd (2017) destaca que operar uma rede sem logs limita significativamente a compreensão dos eventos ocorridos, dificultando a identificação de atividades maliciosas. Gupta (2012) complementa que a análise de eventos em tempo quase real é fundamental para minimizar impactos de incidentes e atender a requisitos de conformidade. Para uma gestão eficiente, a infraestrutura de logging deve ser organizada em camadas funcionais.

Conforme o Kent (2006), esse processo envolve a geração de logs por *hosts* e dispositivos, o armazenamento e a análise por servidores responsáveis pela centralização dos dados, e o monitoramento por meio de interfaces que permitem a visualização e interpretação das informações. Os registros podem ser classificados de acordo com sua finalidade.

A IBM (2025a) destaca os logs de acesso, que registram o comportamento de usuários; os logs de erro, que documentam falhas e incidentes; e os logs de eventos, que representam mudanças de estado no sistema. Independentemente da origem, a padronização é essencial para garantir interoperabilidade.

Segundo Todd (2017), logs bem estruturados devem conter, no mínimo, informações como data, hora, fuso horário, origem, nível de severidade e descrição do evento. A análise de logs tem como objetivo transformar dados brutos em informações úteis para a tomada de decisão. Esse processo envolve etapas como a normalização (*parsing*), que consiste na conversão de dados não estruturados em formatos organizados, facilitando sua interpretação.

De acordo com o Gupta (2012), essa etapa permite a correlação de eventos provenientes de múltiplas fontes, contribuindo para a identificação de ataques mais complexos. Além disso, Todd (2017) destaca a importância da redução de dados desnecessários, visando otimizar armazenamento sem comprometer o valor analítico dos registros.

Como exemplo prático, considere o seguinte registro de um dispositivo de segurança:

Comentado [LM14]: @Lucas Toterol Rodrigues aqui também acho que caberia uma definição do termo

CÓDIGO 1 – Exemplo de um log gerado por um Firewall.

1	date=2026-04-05 time=14:32:10 devname=FW-01 type=traffic
2	subtype=forward srcip=192.168.1.45 dstip=8.8.8.8 action=deny
3	protocol=TCP port=443 msg="Blocked outbound connection"

FONTE: O autor

Nesse caso, é possível identificar que o dispositivo “FW-01” bloqueou uma tentativa de comunicação externa realizada a partir de um endereço IP interno, utilizando o protocolo TCP na porta 443. A análise estruturada desses campos permite compreender o evento registrado e identificar possíveis comportamentos anômalos, sendo fundamental para atividades de monitoramento e segurança.

8 DETECÇÃO DE EVENTOS DE SEGURANÇA

A coleta massiva de dados de monitoramento e logs, discutida nos capítulos anteriores, só atinge seu pleno potencial quando associada a mecanismos eficazes de análise e interpretação. No âmbito da segurança da informação, essa análise converte registros brutos em inteligência acionável contra ameaças cibernéticas. Este capítulo concentra-se na detecção de eventos de segurança a partir da observação de comportamentos anômalos na rede e nos sistemas. Inicialmente, apresentam-se os princípios gerais que norteiam a identificação de atividades maliciosas por meio de Sistemas de Detecção de Intrusão (IDS). Na sequência, são examinados dois vetores de ataque comuns em ambientes Linux e amplamente documentados na literatura: os ataques de força bruta, que visam comprometer credenciais de acesso, e as varreduras de portas (*port scanning*), utilizadas para reconhecimento de infraestrutura. A compreensão desses padrões fundamenta a definição das regras de correlação de eventos que integrarão a proposta do sistema de monitoramento.

8.1 DETECÇÃO DE EVENTOS DE SEGURANÇA E COMO SÃO DETECTADOS?

A detecção de eventos de segurança tem como base a análise de *logs*, que são registros detalhados de tudo o que acontece em um sistema, aplicação ou rede. Conforme discutido no capítulo anterior, registros são essenciais para que os profissionais de TI possam entender o comportamento de seus ambientes e identificar possíveis ameaças. Como explica a IBM (2025a), ferramentas avançadas de análise de *logs* podem até automatizar a detecção de atividades suspeitas, alertando os gestores quando algo anormal ocorre.

Para detectar intrusões, os Sistemas de Detecção de Intrusão (IDS) monitoram o tráfego de rede em busca de padrões maliciosos. Conforme Patel e Sonker (2016), os sistemas baseados em regras (também chamados de detecção por uso indevido) comparam o tráfego com um banco de assinaturas de ataques conhecidos, sendo muito eficazes contra ameaças já catalogadas. Os autores afirmam que “a detecção baseada em regras é muito eficaz contra ataques conhecidos”. No entanto, Brezolin Fernando Nakayama (2024) alertam que “os métodos tradicionais para detectar varreduras de portas são limitados porque se baseiam em regras estáticas e no conhecimento prévio da estrutura da rede”. Para superar essa limitação, Salgueiro e Abreu (2010) propõem o uso de uma linguagem declarativa específica para descrever assinaturas de intrusão que podem se espalhar por vários pacotes de rede, aumentando a capacidade de detecção.

Comentado [LM15]: @Lucas Toterol Rodrigues fiquei curioso com a sugestão do Salgueiro. Isso vai ser aprofundado no trabalho?

8.2 ATAQUES DE FORÇA BRUTA

O ataque de força bruta é uma tentativa repetitiva de adivinhar senhas e credenciais de acesso por meio de tentativa e erro. Como descreve a IBM (2025b), “um ataque de força bruta é um tipo de ciberataque no qual *hackers* tentam obter acesso não autorizado a uma conta ou a dados criptografados por tentativa e erro, testando várias credenciais de *login*”. O serviço SSH (*Secure Shell*), amplamente utilizado para acesso remoto a servidores, é um dos principais alvos desse tipo de ataque. De acordo com a Startup Defense (2025), “um aumento acentuado nas tentativas de *login* com falha é um dos indicadores mais claros de um ataque de força bruta”. Além disso, Park *et al.* (2021) afirmam que a maioria dos *logs* gerados por acessos não autorizados em roteadores de internet é causada por ataques de força bruta SSH, o que permite coletar os endereços IP dos atacantes.

Os riscos de um ataque bem-sucedido são altos. A Fortinet (2026b) destaca que “um ataque de força bruta bem-sucedido pode resultar em acesso não autorizado imediato, permitindo que os atacantes se façam passar pelo usuário [e] roubem dados sensíveis”. Uma vez dentro do sistema, o invasor pode usar os recursos do servidor para fins maliciosos, como participar de *botnets* ou lançar novos ataques. Por isso, monitorar *logs* de falhas de *login* é uma prática de segurança fundamental.

8.3 ATAQUES DE FOOTPRINTING NA REDE (PORT SCAN)

Antes de tentar invadir um sistema, o atacante geralmente realiza um reconhecimento da rede, técnica conhecida como *footprinting*. A principal ferramenta desse reconhecimento é a varredura de portas (*port scan*). Segundo Pittman (2023), “a varredura de portas é o processo de tentar se conectar a várias portas de rede em um dispositivo para determinar quais estão abertas e quais serviços estão sendo executados”. O autor complementa que o primeiro passo de qualquer ataque cibernético é universalmente entendido como o reconhecimento, e que esse reconhecimento quase sempre inclui algum tipo de varredura de portas.

Na prática, o invasor envia pacotes especiais para cada porta do alvo e analisa as respostas. Patel e Sonker (2016) explicam que, por meio dessa técnica, o atacante identifica vulnerabilidades e fraquezas nos pontos de entrada da máquina. Existem diferentes tipos de varredura, como a SYN scan, a FIN scan e a Null scan, que tentam se passar por tráfego normal para evitar a detecção. Brezolin Fernando Nakayama (2024) apresentam um método que detecta varreduras de portas mesmo sem conhecimento prévio da estrutura da rede, o que representa um avanço em relação às abordagens tradicionais.

Comentado [LM16]: @Lucas Toterol Rodrigues top hein - já viu como é?

8.4 A IMPORTÂNCIA DE SABER DETECTAR ESSES ATAQUES

Detectar varreduras de portas e ataques de força bruta logo no início é essencial para evitar danos maiores. Pittman (2023) ressalta que “ao identificar tentativas de varredura de portas, os profissionais de cibersegurança podem tomar medidas proativas para proteger os sistemas e redes antes que um invasor tenha a chance de explorar qualquer vulnerabilidade”. Em outras palavras, a detecção precoce transforma *logs* passivos em uma defesa ativa. Conforme afirmam Brezolin Fernando Nakayama (2024), “a varredura de portas é um primeiro passo em diferentes vetores de ataque. Portanto, é essencial detectar essas varreduras para limitar os seus impactos”.

Além da proteção imediata, a análise de *logs* auxilia no cumprimento de regulamentações como a Lei Geral de Proteção de Dados (LGPD) e padrões do setor financeiro, que exigem rastreabilidade e registros de acesso. Como destaca o Kent (2006), a análise regular de *logs* ajuda os profissionais de TI a entender melhor como seus sistemas estão funcionando, melhorar o desempenho e aumentar a segurança. Em resumo, saber detectar esses ataques não é apenas uma questão técnica, é uma necessidade estratégica para qualquer organização que queira proteger seus dados e sua reputação.

9 COMUNICAÇÃO E ARQUITETURA

O modelo cliente-servidor estrutura a comunicação entre programas em dois papéis: o cliente é o responsável por iniciar a requisição e o servidor é responsável por processá-la e retornar uma resposta. Nesse modelo, a comunicação entre cliente e servidor ocorre por meio de uma API que atua como intermediária: quando o cliente quer algo, ele faz uma requisição seguindo as regras definidas pela API, o servidor recebe essa requisição, a processa e devolve uma resposta. Essas regras determinam como os dados devem ser enviados, em qual formato e quais operações são permitidas, garantindo que os dois lados se comuniquem de forma padronizada e organizada.

De acordo com Oliveira (2016, p. 24), os principais componentes desse modelo são os servidores, responsáveis por oferecer um conjunto de serviços, como servidores de impressão, de arquivos e com *web services*; os clientes, que usam os serviços disponibilizados nos servidores, sendo tipicamente subsistemas executados independentemente que, embora informados sobre os servidores disponíveis, desconhecem a existência de outros clientes, destacando-se que seus processos são separados; e uma rede que viabilize o acesso por parte dos clientes aos serviços, componente este desnecessário apenas quando cliente e servidor encontram-se na mesma máquina. Ressalta-se, no entanto, que sistemas que fazem uso de uma arquitetura cliente-servidor utilizam um sistema distribuído.

JSON é um formato de texto para organizar dados. Não pertence a nenhuma linguagem de programação (define só estrutura), então qualquer sistema consegue ler o que outro escreveu. Segundo a IBM (2023), os dados ficam em pares de chave e valor, o que facilita tanto a leitura por máquina quanto a interpretação de quem está desenvolvendo. Essa padronização também ajuda na troca de informações entre sistemas diferentes, já que todos seguem a mesma lógica de estrutura.

Sua estrutura leve e compacta o torna mais eficiente que o XML, que carrega mais marcações e ocupa mais espaço para representar a mesma informação. Essa diferença de peso aparece especialmente em sistemas que fazem muitas requisições, onde cada *kilobyte* a menos no *payload* conta. Em aplicações modernas, como APIs e sistemas distribuídos, essa economia de dados contribui diretamente para melhor desempenho e menor consumo de recursos.

HTTP é o protocolo que move dados na *web*. O modelo é simples: cliente pede, servidor responde. Esse funcionamento é a base da *web*, como destaca a MDN Web Docs (2026b). Cada troca é independente, o que facilita escalar sistemas sem aumentar a complexidade. Isso permite que aplicações atendam um grande número de requisições sem precisar manter informações sobre interações anteriores.

Os métodos GET, POST, PUT e DELETE indicam o tipo de operação. Os cabeçalhos carregam informações sobre formato dos dados e autenticação. Os códigos de

Comentado [LM17]: @Lucas Toterol Rodrigues muito legal

Comentado [LM18]: @Lucas Toterol Rodrigues não sei se caberia mencionar o HTTPS aqui também

status (200 para sucesso, 404 para não encontrado) dizem ao cliente o que aconteceu com cada pedido, tornando o diagnóstico de erros mais direto. De acordo com a Fielding *et al.* (2022), esses elementos fazem parte da padronização do protocolo HTTP, garantindo uma comunicação consistente entre sistemas.

APIs conectam sistemas diferentes sem expor como funcionam por dentro. Elas recebem requisições, processam e respondem, mantendo a complexidade encapsulada e deixando os sistemas mais fáceis de manter. Isso permite que diferentes partes de um sistema evoluam de forma independente, sem impactar diretamente outras áreas.

Na prática, APIs definem como as requisições e respostas devem ser estruturadas: definem o que pode ser pedido, em qual formato e o que será devolvido. Como descrito na MDN Web Docs (2026a), esse modelo permite que partes diferentes de um sistema sejam desenvolvidas de forma independente. Um aplicativo móvel pode puxar dados de um *backend* sem saber nada sobre o banco de dados, o que facilita a integração entre diferentes tecnologias e plataformas.

O Zabbix mostra bem como essas três tecnologias funcionam juntas. De acordo com a documentação oficial, a ferramenta usa APIs baseadas em HTTP para comunicação, enquanto os dados são transmitidos em JSON (ZABBIX, 2026). Isso permite coleta e análise em tempo real, com integração a outros sistemas sem exigir adaptações complexas em cada ponta. Na prática, isso possibilita monitorar servidores, redes e aplicações de forma centralizada, além de automatizar alertas e acompanhar métricas importantes em tempo real. O resultado é um fluxo de monitoramento contínuo, onde as informações trafegam em formato leve e são acessíveis por qualquer sistema que fale HTTP.

10 APLICAÇÃO DOS CONCEITOS NO PLANEJAMENTO DO SISTEMA DE MONITORAMENTO

A proposta de um sistema de monitoramento, conforme delineada no Capítulo 1, pressupõe a capacidade de coletar métricas operacionais, armazenar *logs*, gerar alertas e identificar eventos relevantes a partir da análise desses dados. Entretanto, a implementação dessas funcionalidades não é trivial e envolve uma série de decisões arquiteturais que dependem diretamente dos conceitos discutidos na fundamentação teórica.

Neste capítulo, apresenta-se o planejamento da solução proposta, evidenciando como os conceitos abordados nos Capítulos 2 a 9 fundamentam as escolhas realizadas. Mais do que descrever a implementação, busca-se demonstrar que decisões como modelo de coleta, arquitetura de comunicação, estrutura de armazenamento e mecanismos de detecção de eventos só se tornam compreensíveis à luz da teoria previamente apresentada.

10.1 VISÃO GERAL DA SOLUÇÃO PROPOSTA

O sistema adota uma arquitetura centralizada, na qual um servidor único é responsável por receber, processar e armazenar dados provenientes de múltiplos clientes monitorados. Essa escolha está diretamente relacionada ao modelo agente-servidor discutido no Capítulo 6, no qual agentes distribuídos coletam informações localmente e as encaminham a um gerenciador central.

Apesar de arquiteturas distribuídas oferecerem maior escalabilidade e tolerância a falhas, optou-se por um modelo centralizado devido ao escopo controlado do projeto e à redução da complexidade de implementação. Essa decisão, no entanto, implica um ponto único de falha, o que será discutido posteriormente como limitação da solução.

Os clientes executam agentes de monitoramento responsáveis por coletar dados do sistema operacional e enviá-los ao servidor. Essa abordagem segue o princípio de agentes autônomos descrito por Wooldridge e Jennings (1995), nos quais entidades são capazes de perceber o ambiente e agir de forma proativa.

No que se refere à comunicação, adotou-se o modelo *push*, no qual os próprios agentes enviam os dados ao servidor em intervalos regulares. Essa escolha está alinhada com os conceitos apresentados no Capítulo 4, especialmente no que diz respeito ao balanceamento entre frequência de coleta e impacto na rede. Diferentemente do modelo *polling*, o *push* reduz requisições desnecessárias, porém pode gerar picos de carga no servidor caso múltiplos agentes enviem dados simultaneamente.

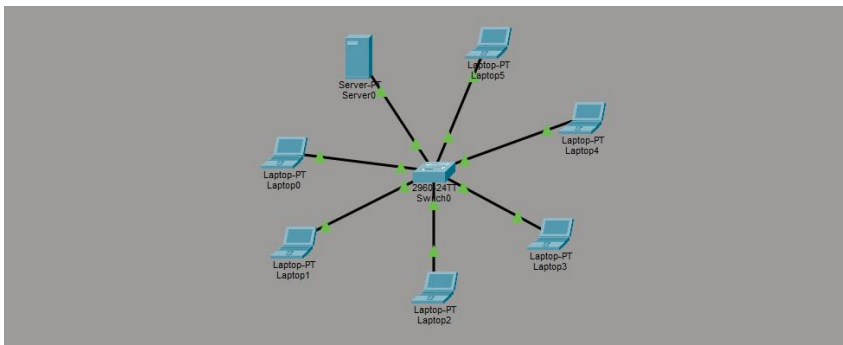
10.2 ARQUITETURA FÍSICA E LÓGICA

A arquitetura física do sistema é composta por um servidor central e sete clientes, todos conectados a um *switch* não gerenciável em topologia estrela. A rede utiliza a faixa 10.10.10.0/27, permitindo até 30 endereços IP utilizáveis, com atribuição dinâmica realizada por meio de um serviço DHCP executado no próprio servidor.

Essa configuração simplifica o gerenciamento da rede, mas limita a capacidade de controle sobre o tráfego, uma vez que o *switch* utilizado não permite segmentação ou priorização de pacotes, o que pode impactar o desempenho em cenários de maior carga.

A Figura 1 ilustra a disposição dos componentes na rede local. O servidor central (10.10.10.2) executa os serviços de *backend*, banco de dados, DHCP e *proxy* reverso. Os sete clientes (10.10.10.3 a 10.10.10.9) executam o agente coletor. Todos os dispositivos são interligados por um *switch* não gerenciável, caracterizando uma topologia estrela.

FIGURA 1 – Topologia lógica do sistema de monitoramento.



FONTE: Os autores (2026).

No plano lógico, o servidor é estruturado utilizando contêineres Docker. A containerização consiste no empacotamento de uma aplicação e suas dependências em unidades isoladas e portáteis, garantindo que o ambiente de execução seja reproduzível independentemente da máquina hospedeira. Essa abordagem reduz conflitos de dependência e facilita a implantação dos componentes do sistema, embora não elimine a necessidade de gerenciamento de recursos, especialmente em ambientes com limitação de CPU e memória.

Os principais componentes do servidor são:

- **Nginx:** atua como *proxy* reverso, recebendo requisições HTTP e encaminhando-as ao *backend*. Essa camada permite futura adoção de HTTPS e centraliza o controle de acesso.

- **Backend (Python/Flask):** responsável por expor a API REST, validar requisições e processar os dados recebidos. A validação é realizada por meio de um *token* estático, o que simplifica a implementação, mas não garante autenticação robusta entre cliente e servidor.
- **PostgreSQL:** utilizado para armazenamento de métricas e *logs*. A escolha por um banco relacional se deve à necessidade de consultas estruturadas, embora isso possa limitar a escalabilidade em cenários com alto volume de escrita contínua.
- **Dashboard (HTML/CSS/JS + Chart.js):** responsável pela visualização dos dados, consumindo informações via API.

10.3 COLETA DE DADOS E AGENTES DE MONITORAMENTO

A coleta de dados implementada pelos agentes segue dois modelos distintos: coleta inicial (*discovery*) e coleta contínua de métricas.

A coleta inicial tem como objetivo estabelecer uma linha de base do sistema monitorado, incluindo informações de *hardware*, sistema operacional e *software* instalado. Essa etapa é essencial para contextualizar as métricas coletadas posteriormente.

Já a coleta contínua ocorre em intervalos de 5 a 10 segundos, capturando métricas como uso de CPU, memória, disco e rede. A escolha desse intervalo reflete um compromisso entre granularidade dos dados e sobrecarga no sistema, conforme discutido no Capítulo 2.

A seleção das métricas baseia-se no método USE (*Utilization, Saturation, Errors*), proposto por Gregg (2013), que orienta a análise de desempenho a partir da utilização de recursos, níveis de saturação e ocorrência de erros.

Além das métricas, os agentes coletam *logs* do sistema operacional, permitindo a análise de eventos e a identificação de comportamentos suspeitos. A centralização desses *logs* segue as recomendações de Kent (2006), que destaca sua importância para auditoria e investigação de incidentes.

Os dados são enviados ao servidor via requisições HTTP POST em formato JSON. Embora essa abordagem seja simples e amplamente suportada, ela não garante confidencialidade ou integridade dos dados em sua forma básica, sendo necessária a adoção de HTTPS em versões futuras.

10.4 ARMAZENAMENTO E ESTRUTURAÇÃO DOS DADOS

O sistema utiliza o PostgreSQL como banco de dados principal, estruturando as informações em tabelas relacionais para *hosts*, métricas, *logs* e alertas.

A utilização do tipo JSONB permite armazenar dados semiestruturados, o que oferece flexibilidade na evolução do sistema sem necessidade de alterações frequentes no *schema*.

Entretanto, o uso de um banco relacional único pode se tornar um gargalo em cenários com alta taxa de ingestão de dados, especialmente devido à natureza contínua das métricas coletadas.

Para mitigar o crescimento descontrolado do banco, foi definida uma política de retenção de dados de 7 dias. Esse período foi escolhido por equilibrar a necessidade de análise histórica de curto prazo, suficiente para identificar tendências semanais e investigar incidentes recentes, com a limitação de armazenamento disponível no ambiente acadêmico, evitando o crescimento excessivo da base de dados.

10.5 PROCESSAMENTO DE EVENTOS E ALERTAS

O processamento dos dados ocorre por meio de regras definidas no *backend*, que avaliam tanto métricas quanto *logs*.

No contexto de monitoramento de infraestrutura, são utilizados limiares (*thresholds*) para geração de alertas, como uso elevado de CPU ou baixo espaço em disco. Essa abordagem é simples e eficaz, mas depende de calibração adequada para evitar excesso de notificações.

No que se refere à segurança, a análise baseia-se em padrões conhecidos identificados nos *logs*, como múltiplas tentativas de autenticação malsucedidas (indicando possível ataque de força bruta) ou conexões em múltiplas portas (indicando varredura de portas). Conforme discutido no Capítulo 8, sistemas baseados exclusivamente em regras são eficazes contra ameaças conhecidas, mas podem falhar na identificação de comportamentos inéditos. A opção por essa abordagem simplificada é compatível com o objetivo de prova de conceito do presente trabalho, reduzindo a complexidade de implementação sem comprometer a validação dos princípios de detecção estudados.

10.6 VISUALIZAÇÃO E DASHBOARD

O *dashboard web* apresenta os dados coletados por meio de gráficos e tabelas, permitindo a visualização do estado atual do sistema e a análise de tendências.

As principais funcionalidades incluem:

- Visualização de métricas em séries temporais;
- Exibição de *logs* recentes;
- Listagem de alertas ativos;
- *Status* dos *hosts* monitorados

Embora a interface permita a análise operacional do ambiente, sua eficácia depende diretamente da qualidade dos dados coletados e das regras definidas no *backend*.

10.7 LIMITAÇÕES DA SOLUÇÃO

A solução proposta apresenta limitações que decorrem das decisões arquiteturais adotadas:

- Presença de ponto único de falha no servidor central;
- Ausência de criptografia na comunicação inicial (HTTP);
- Autenticação simplificada baseada em *token* estático;
- Inexistência de mecanismos robustos de *retry* nos agentes;
- Ausência de sistema de filas para processamento assíncrono;
- Baixa escalabilidade para ambientes com grande número de clientes.

Essas limitações refletem o caráter acadêmico do projeto e indicam caminhos claros para evolução futura, especialmente no que diz respeito à segurança, escalabilidade e resiliência do sistema.

O planejamento aqui descrito evidencia que cada componente do sistema de monitoramento encontra respaldo direto nos capítulos teóricos precedentes. A compreensão dos conceitos de métricas, *logs*, observabilidade, arquitetura cliente-servidor e segurança da informação mostrou-se imprescindível para a tomada de decisões arquiteturais conscientes e para a construção de uma solução que, embora simplificada, é funcional e didaticamente relevante. Os próximos capítulos detalharão a implementação prática dessas especificações e os resultados obtidos com a execução do sistema em ambiente controlado.

Comentado [LM19]: @Lucas Toterol Rodrigues e aí cade os proximos capitulos? Quando lanca a proxima "temporada" ?

11 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo analisar os fundamentos de monitoramento de sistemas e segurança da informação, aplicando-os na concepção de uma proposta arquitetural para um sistema de monitoramento voltado a ambientes Linux. A partir da revisão teórica realizada, foi possível estruturar uma solução que integra a coleta de métricas de infraestrutura com a análise de registros de eventos.

O objetivo geral foi atendido no nível de especificação, por meio da definição de uma arquitetura cliente-servidor composta por agentes de coleta e um servidor central responsável pelo processamento, armazenamento e visualização das informações. Foram delimitados os principais componentes do sistema, os fluxos de dados e os mecanismos básicos de detecção de eventos de segurança baseados em regras.

A fundamentação teórica mostrou-se essencial para orientar as decisões arquitetais. O uso do método USE contribuiu para a definição das métricas de desempenho, enquanto os conceitos de observabilidade reforçaram a importância da correlação entre métricas e logs. As diretrizes de segurança analisadas permitiram a identificação de padrões simples de comportamento anômalo, ainda que limitados a abordagens determinísticas.

Apesar da coerência da proposta, o sistema apresenta limitações relevantes decorrentes de sua natureza conceitual. A arquitetura centralizada implica a existência de um ponto único de falha, e a ausência de mecanismos de comunicação segura e processamento assíncrono pode comprometer a confiabilidade e a escalabilidade em cenários reais. Adicionalmente, a dependência de regras estáticas restringe a capacidade de detecção de anomalias mais complexas.

Como trabalhos futuros, recomenda-se a implementação prática da arquitetura proposta, seguida de testes de desempenho e escalabilidade em ambientes controlados. Sugere-se também a evolução da arquitetura para um modelo orientado a eventos, com uso de *message brokers* para desacoplamento da ingestão de dados, além da adoção de comunicação segura e mecanismos mais avançados de detecção de anomalias.

Dessa forma, o trabalho cumpre seu papel ao consolidar a fundamentação teórica e demonstrar sua aplicação na concepção de um sistema de monitoramento, fornecendo uma base estruturada para futuras implementações e investigações na área.

REFERÊNCIAS

- ACETO, G. *et al.* Cloud monitoring: A survey. **Computer Networks**, v. 57, p. 2093–2115, 2013. Disponível em: <https://www.sciencedirect.com/science/article/abs/pii/S1389128613001084>. Acesso em: 09 abr. 2026.
- AMAZON WEB SERVICES. **Best Practices for Monitoring Amazon EC2 Instances**. 2023. Disponível em: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/monitoring_best_practices.html. Acesso em: 09 abr. 2026.
- AMAZON WEB SERVICES. **What is an Event-Driven Architecture?** 2025. Disponível em: <https://aws.amazon.com/event-driven-architecture/>. Acesso em: 08 abr. 2026.
- AMAZON WEB SERVICES. **O que é armazenamento de banco de dados? | Explicação do armazenamento de banco de dados**. 2026. Disponível em: <https://aws.amazon.com/pt/database-storage/>. Acesso em: 08 abr. 2026.
- BARRETT, R.; KING, T.; KORB, A. **Performance Analysis and Tuning on Modern Systems**. [s.n.], 2016. Disponível em: <https://dl.acm.org/doi/book/10.1145/2851613>. Acesso em: 09 abr. 2026.
- BEYER, B. *et al.* **Site Reliability Engineering: How Google Runs Production Systems**. O'Reilly Media, 2016. Disponível em: <https://sre.google/sre-book/monitoring-distributed-systems/>. Acesso em: 09 abr. 2026.
- BREZOLIN FERNANDO NAKAYAMA, M. N. U. Detecção de varreduras de portas pela análise inteligente de tráfego de rede iot. In: **Anais do XXIII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg)**. Porto Alegre, RS: SBC, 2024. Disponível em: <https://sol.sbc.org.br/index.php/sbseg/article/view/30031>. Acesso em: 09 abr. 2026.
- CABALLERO, M. A. G. **Impacto da Utilização de Dashboards na Tomada de Decisão Estratégica: Um Estudo Empírico**. 2024. Disponível em: <https://repositorio.ufms.br/jspui/retrieve/c4a65b9d-4c73-45bd-8741-591878383b00/13302.pdf>. Acesso em: 09 abr. 2026.
- CAMPOS, L. L. A. O. **Desenvolvimento de um Sistema de Dashboard Dinâmico para Acompanhamento das Ações Promovidas no Projeto Meninas na Ciência da Computação**. 2023. Disponível em: https://repositorio.ufpb.br/jspui/bitstream/123456789/32550/1/Laura%20Let%c3%adcia%20Ara%c3%bajo%20Oliveira%20Campos_TCC.pdf. Acesso em: 02 abr. 2026.
- CASE, J. *et al.* **Simple Network Management Protocol (SNMP)**. [S.l.], 2002. Disponível em: <https://datatracker.ietf.org/doc/html/rfc3411>. Acesso em: 05 abr. 2026.
- CEDERBERG, A. **Observability in Microservices: A Study on how to achieve Observability in a Microservice Architecture**. 74 p. Dissertação (Dissertação (Mestrado em Ciência da Computação)) — KTH Royal Institute of Technology, Estocolmo, 2021. Disponível em: <https://www.proquest.com/docview/2561141755>. Acesso em: 04 abr. 2026.

CHINA, C. R. **Observability vs Monitoring**. 2024. Disponível em: <https://www.ibm.com/topics/observability>. Acesso em: 09 abr. 2026.

DAVENPORT, T. H. Competing on analytics. **Harvard Business Review**, v. 84, p. 98–107, 2006. Disponível em: <https://hbr.org/2006/01/competing-on-analytics>. Acesso em: 09 abr. 2026.

DIO. **Como funciona uma API**. 2026. Disponível em: <https://www.dio.me/articles/como-funciona-uma-api>. Acesso em: 04 abr. 2026.

FERREIRA, L. **Monitoramento contínuo de redes de computadores**. Dissertação (Mestrado) — Universidade Estadual Paulista (UNESP), 2020. Disponível em: <https://repositorio.unesp.br/entities/publication/2ee78ae7-943d-4257-8eea-ab3ab33a5005>. Acesso em: 05 abr. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. [S.l.], 2022. Disponível em: <https://www.rfc-editor.org/rfc/rfc9110>. Acesso em: 09 abr. 2026.

FORTINET. **Network Monitoring**. 2026. Disponível em: <https://www.fortinet.com/br/resources/cyberglossary/network-monitoring>. Acesso em: 09 abr. 2026.

FORTINET. **O que é um ataque de força bruta?** 2026. Disponível em: <https://www.fortinet.com/br/resources/cyberglossary/brute-force-attack>. Acesso em: 09 abr. 2026.

GOOGLE CLOUD. **Alerting overview**. 2023. Disponível em: <https://cloud.google.com/stackdriver/docs/alerting>. Acesso em: 09 abr. 2026.

GREGG, B. **The USE Method of Performance Analysis**. 2013. Disponível em: <https://freebsdoundation.org/wp-content/uploads/2014/07/The-USE-Method.pdf>. Acesso em: 09 abr. 2026.

GREGG, B. **Systems Performance: Enterprise and the Cloud**. 2. ed. Addison-Wesley, 2020. Disponível em: <https://www.brendangregg.com/systems-performance-2nd-edition-book.html>. Acesso em: 09 abr. 2026.

GUPTA, S. **Logging and Monitoring to Detect Network Intrusions and Compliance Violations in the Environment**. [S.l.], 2012. Disponível em: <https://www.sans.org/white-papers/33985>. Acesso em: 09 abr. 2026.

IBM. **Server Monitoring Concepts**. 2022. Disponível em: <https://www.ibm.com/topics/server-monitoring>. Acesso em: 09 abr. 2026.

IBM. **What is IT Monitoring?** 2022. Disponível em: <https://www.ibm.com/topics/it-monitoring>. Acesso em: 09 abr. 2026.

IBM. **Formato JSON (JavaScript Object Notation)**. 2023. Disponível em: <https://www.ibm.com/docs/en/baw/23.0.x?topic=formats-javascript-object-notation-json-format>. Acesso em: 09 abr. 2026.

IBM. **O que é análise de logs?** 2025. Disponível em: <https://www.ibm.com/br-pt/think/topics/log-analysis>. Acesso em: 09 abr. 2026.

- IBM. **O que é um ataque de força bruta?** 2025. Disponível em: <https://www.ibm.com/br-pt/think/topics/brute-force-attack>. Acesso em: 09 abr. 2026.
- KENT, M. S. K. **Guide to Computer Security Log Management**. [S.l.], 2006. Disponível em: <https://csrc.nist.gov/pubs/sp/800/92/final>. Acesso em: 09 abr. 2026.
- LIMA, R. **Planejamento de capacidade em ambientes de TI**. Dissertação (Mestrado) — Centro Paula Souza (CPS), 2021. Disponível em: <https://ric.cps.sp.gov.br/handle/123456789/14883>. Acesso em: 05 abr. 2026.
- MAJORS, C.; FONG-JONES, L.; MIRANDA, G. **Observability Engineering: Achieving Production Excellence**. 1. ed. O'Reilly Media, 2022. 320 p. Disponível em: <https://www.oreilly.com/library/view/observability-engineering/9781492076438/>. Acesso em: 04 abr. 2026.
- MDN WEB DOCS. **Interface de Programação de Aplicações (API)**. 2026. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/API>. Acesso em: 09 abr. 2026.
- MDN WEB DOCS. **Visão geral do HTTP**. 2026. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Overview>. Acesso em: 09 abr. 2026.
- MICROSOFT. **Event-driven architecture style**. 2026. Azure Architecture Center | Microsoft Learn. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven/>. Acesso em: 08 abr. 2026.
- OLIVEIRA, K. S. F. d. **Arquitetura de Software**. São Leopoldo: Editora Unisinos, 2016.
- OpenTelemetry. **OpenTelemetry Specification**. 2023. Disponível em: <https://opentelemetry.io/docs/specs/>. Acesso em: 04 abr. 2026.
- PARK, J.; KIM, J.; PARK, N. Network log-based ssh brute-force attack detection model. **Computers, Materials & Continua**, v. 68, n. 1, p. 1083–1098, 2021. Disponível em: https://www.researchgate.net/profile/Brij-B-Gupta/publication/350517340_Network_Log-Based_SSH_Brute-Force_Attack_Detection_Model/links/606af84c92851c91b1a516ef/Network-Log-Based-SSH-Brute-Force-Attack-Detection-Model.pdf. Acesso em: 09 abr. 2026.
- PATEL, S.; SONKER, A. Efficient port scan detection rules for intrusion detection system. In: **Proceedings of the International Conference on Recent Advances in Computing and Communications**. [S.l.]: Science & Engineering Research Support Society (SERSC), 2016. p. 1–12. Título inferido do conteúdo do artigo. Disponível no repositório do autor.
- PITTMAN, J. M. Machine learning and port scans: A systematic review. **arXiv preprint**, 2023. Disponível em: <https://arxiv.org/abs/2301.13581>. Acesso em: 09 abr. 2026.
- RIOS, V. **Entendendo Polling: Técnicas, Implementações e Alternativas para Comunicação em Tempo Real**. 2024. DEV Community. Disponível em: <https://dev.to/vitorrios/entendendo-polling-tecnicas-implementacoes-e-alternativas-para-comunicacao-em-tempo-real>. Acesso em: 08 abr. 2026.

ROLIM, D. A. de A. **Dashboards para Desenvolvimento de Aplicações e Visualização de Dados para Plataformas de Cidades Inteligentes**. 2020. Disponível em: <https://repositorio.ufrn.br/bitstreams/f927b3be-36dd-433d-8394-f929557920fb/download>. Acesso em: 09 abr. 2026.

RUSSELL, S.; NORVIG, P. **Inteligência Artificial: Uma Abordagem Moderna**. Rio de Janeiro: Editora Elsevier, 2013.

SALGUEIRO, P.; ABREU, S. Nemode: A declarative system for network intrusion detection. In: **Proceedings of the 3rd International Conference on Security of Information and Networks (SIN 2010)**. New York, NY, USA: ACM, 2010. Disponível em: <https://dspace.uevora.pt/rdpc/handle/10174/32367>. Acesso em: 09 abr. 2026.

SANTOS, R. **Monitoramento de redes de computadores utilizando SNMP**. Dissertação (Mestrado) — Universidade Tecnológica Federal do Paraná (UTFPR), 2019. Disponível em: <https://repositorio.utfpr.edu.br/jspui/handle/1/17092>. Acesso em: 05 abr. 2026.

SIGELMAN, S. *et al.* **Dapper, a Large-Scale Distributed Systems Tracing Infrastructure**. [S.l.], 2010. Disponível em: <https://research.google/pubs/pub36356/>. Acesso em: 04 abr. 2026.

SOUZA, J.; OLIVEIRA, M. Nagios: monitorando a saúde de serviços e sistemas. **Revista Interface Tecnológica**, 2020. Disponível em: <https://revista.fatectq.edu.br/interfacetecnologica/article/view/1112>. Acesso em: 05 abr. 2026.

Startup Defense. **SSH Brute Force**. 2025. Disponível em: <https://www.startupdefense.io/cyberattacks/ssh-brute-force>. Acesso em: 09 abr. 2026.

SUSNJARA, S.; SMALLEY, I. **O que é armazenamento de dados?** 2025. Disponível em: <https://www.ibm.com/br-pt/think/topics/data-storage>. Acesso em: 08 abr. 2026.

TANENBAUM, A. S.; BOS, H. **Sistemas Operacionais Modernos**. 4. ed. Pearson Education do Brasil, 2015. 1136 p. Disponível em: [https://www.kufunda.net/publicdocs/Sistemas%20Operacionais%20Modernos%20\(Andrew%20S.%20Tanenbaum,%20Herbert%20Bos\).pdf](https://www.kufunda.net/publicdocs/Sistemas%20Operacionais%20Modernos%20(Andrew%20S.%20Tanenbaum,%20Herbert%20Bos).pdf). Acesso em: 09 abr. 2026.

TODD, B. **Creating a Logging Infrastructure**. [S.l.], 2017. Disponível em: <https://www.sans.org/white-papers/38130>. Acesso em: 09 abr. 2026.

WOOLDRIDGE, M.; JENNINGS, N. R. Intelligent agents: theory and practice. **The Knowledge Engineering Review**, Cambridge, UK, v. 10, n. 2, p. 115–152, 1995.

ZABBIX. **Zabbix: Monitoramento de código aberto para servidores, redes e aplicações**. 2026. Disponível em: <https://www.zabbix.com>. Acesso em: 09 abr. 2026.